

0 地基：先让 AI 懂你的项目

这一步省了，后面所有提示词都会打折。在项目根目录建 `.cursorrules`（其他工具对应各自的规则文件）。

```
# 项目上下文
技术栈: Vue 3.4 + TypeScript + Vite + Pinia + Element Plus
目录: src/components 通用组件 / views 页面 / api 接口层 / types 类型
代码规范:
- 组件一律用 <script setup lang="ts">
- 状态管理只用 Pinia
- 样式用 scoped + CSS 变量, 禁止行内样式
- 禁止 any, 无法推断时用 unknown + 类型守卫
- 组件名 PascalCase, 文件与组件同名
- 接口请求统一走 src/api, 禁止在组件里直接 axios
禁止事项:
- 不要生成 .sync / this.$refs / this.$emit (Vue2 语法)
- 不要引入新的第三方库
- 不要写无意义的防御性判空
- 不要用 mock 数据填充业务逻辑
输出要求:
- 先说实现思路 (≤3 句), 再给代码
- 关键处写中文注释
- 生成后自己 review 一遍, 列出你认为有风险的地方
```

三类必须喂给 AI 的上下文: ① 一个你写得最好的同类组件（效果最显著，比写 500 字描述有用）② 接口返回的数据结构 / 类型定义 ③ 团队已有的工具函数（避免重复造轮子）

1 六大组件场景模板

1 表单组件

```
参考 @components/UserForm.vue 的写法, 生成「业务名」表单组件。
字段: 字段名 (类型 / 校验规则 / 控件) 布局: 两列栅格, label 宽 100px
行为: 提交前整表校验; 提交中按钮 loading; 重置仅清空不触发校验; v-model 双向绑定整个表单对象
约束:
- 校验用 async-validator 风格, 错误提示具体化 (「请输入手机号」而不是「不能为空»)
- 不要 .sync / this.$emit - 不要引入新 UI 库
输出: 组件代码 + 使用示例
```

⚠ 必查 ① 自定义控件（日期 / 上传）的 resetFields 是否真的清空 ② 校验规则是否覆盖全部必填项 ③ 表单对象是否因解构 props 丢了响应式

2 数据表格

```
生成「业务名」数据表格组件, 字段: ...
行为: 分页 (前端 / 后端)、多选、行操作、空状态、loading
约束:
- 必须设 row-key (用唯一 id, 绝不用 index)
- 分页参数与父组件 v-model:page 同步, 不要内部私有状态
- selection 变化 emit 出去, 组件内不处理业务逻辑
- 长文本列用 show-overflow-tooltip, 不要换行撑高 - 超过 10 列改成配置化 columns
```

⚠ 必查 ① row-key 缺失导致选中态错乱 ② 分页状态与父组件不同步 ③ loading 未覆盖翻页 ④ 跨页多选丢失

3 弹窗 / 抽屉

```
生成「业务名」弹窗 (或 Drawer) 组件。
行为: v-model 控制显隐; 有未保存改动时关闭需二次确认; 确认按钮 loading; 关闭后销毁内部状态
约束:
- 用 v-model 控制显示, 不要用 v-if 硬控 (会丢动画、残留上一次的表单数据)
- 关闭时清空校验状态 - 内容溢出时内部滚动, 不要撑破页面
- 不在弹窗内直接调接口, emit 给父组件
```

⚠ 必查 ① 关闭后残留上次数据 (最高频) ② loading 期间可重复点击 ③ 二次确认被写成 window.confirm

4 列表 / 下拉加载

生成「业务名」无限滚动列表（下拉刷新）。

行为：触底加载下一页、无更多提示、首屏 loading、失败可重试、空状态

约束：

- 用 IntersectionObserver，不要监听 scroll 事件 - 组件卸载时移除 observer
- 加载中加锁，禁止重复触发 - 分页游标由父组件管理 - 不要用 setTimeout 模拟延迟

⚠ 必查 ① 触底重复请求 ② observer 未销毁（内存泄漏） ③ 快速滚动导致数据顺序错乱 ④ 无更多时仍在请求

5 图表容器

生成「业务名」图表容器组件，数据由 props 传入。

约束：

- 用 shallowRef 存 echarts 实例，别用 ref (Proxy 会让 echarts 卡顿甚至报错)
- 组件卸载时必须 dispose() - resize 用 ResizeObserver + 防抖，卸载时移除监听
- 容器固定高度，不能靠内容撑开 - 数据更新用 setOption(option, false)，不要重新 init

⚠ 必查 ① 未 dispose（路由反复切换后页面卡死） ② resize 未防抖 ③ 数据结构变化时残留旧 series

6 业务卡片

生成「业务名」业务卡片：标题、状态标签、描述、操作区。

约束：

- 状态与文案的映射抽成常量对象，不要散在模板里 - 操作区用 slot 暴露，组件内不写业务逻辑
- 文本截断用 line-clamp，不要 JS 截字符串 - 响应式：≤768px 纵向排列
- 状态标签带语义色，不要只用文字区分状态

⚠ 必查 ① 状态映射写死在 template ② 移动端溢出 ③ 卡片整体可点时与内部按钮的点击穿透

2 调试与重构场景

7 报错定位

【报错信息】...

【相关代码】...

【我已经试过】...

请按顺序回答：

1. 最可能的 3 个原因，按概率排序
 2. 每个原因的验证方法（我怎么确认是它）
 3. 修复方案，优先不改动架构的最小改动
- 不要一上来就重写，先定位。

8 Vue2 → Vue3 迁移

把下面的 Vue2 组件迁移到 Vue3 + <script setup lang="ts">。

【原组件】...

要求：

- this.xxx → ref / reactive / composable
- 生命周期：beforeDestroy→onBeforeUnmount, destroyed→onUnmounted
- \$refs → ref 变量 + 同名绑定 - \$children / \$listeners / .sync 一律重写 (Vue3 已移除)
- filters → 计算属性或方法 - 保留原注释，迁移点用 // MIGRATE: 标注
- 最后列出哪些第三方库行为可能不一致

9 TypeScript 类型补全

为下面的 JS 文件补 TypeScript 类型。

【代码】...

要求：

- 函数入参、返回值显式标注 - 接口返回结构抽成 interface 放 src/types
- 不用 any；无法推断的用 unknown + 类型守卫并注释原因
- 泛型优先于 any[]，联合类型优先于宽泛的 string
- 补完跑 tsc --noEmit，逐个修掉报错再给我

⚠ AI 补不准的三类 复杂泛型推断 · 第三方库类型扩展 · 依赖运行时判断的联合类型收窄

3 万能提示词骨架（填空即用）

【角色】你是一个资深前端工程师，熟悉 <技术栈>。
【上下文】项目背景：一句话。现有代码：贴一个同类范例。
【任务】实现 具体功能。
【约束】
 必须：硬性要求
 禁止：你踩过的坑 / 团队规范红线
【输出】先说思路（≤3 句），再给代码，最后列出你认为有风险的地方。
【自检】生成后按 检查清单 逐条核对，没通过的自己改。

4 三条通用原则

01
先给上下文，再给任务

提示词质量的 80% 在上下文里。贴一个你写得最好的组件，效果远好于写 500 字描述。

02
禁令比要求有效

「不要用 .sync」比「请用 Vue3 语法」有效得多——AI 对「不要做什么」的服从度明显更高。

03
生成后让它自己 review

加一句「列你觉得有风险的地方」，它能自己抓出一半问题，剩下的一半才需要你。

5 高频翻车点速查

场景	AI 常犯	一句话规避
任何组件	混入 Vue2 语法 (.sync / this.\$refs)	.cursorsrules 里写死禁令，比每次提醒管用
任何组件	大量无意义防御性判空	提示词里加「不做无意义防御」
任何组件	重复实现已有的工具函数	把 utils 目录喂给它，或贴函数签名
列表 / 表格	用 index 当 key / 漏 row-key	提示词里明写「用唯一 id，绝不用 index」
响应式	解构 props 导致响应式丢失	要求用 toRefs 或直接 props.xxx 访问
图表 / 监听	未 dispose、未移除 observer	强制要求「卸载时清理」并写在输出要求里
样式	响应式断点写死 px	要求用 CSS 变量 + clamp()，禁止写死断点
类型	大量 any 蒙混过关	要求「补完跑 tsc --noEmit，报错修完再给」
重构	顺手改掉无关代码，diff 爆炸	明确「只改涉及的部分，不要顺手优化」

提示词不是背的，是攒的。同一个需求问两次、对比差异，你很快就能摸清一个模型的脾气。最贵的一步永远是人 review：AI 生成的代码至少要过一遍响应式、生命周期、内存释放、边界状态这四关。